COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	07/31/26	**DATE SUBMITTED:**	07/31/26
NAME:	Hare, Andrei Angelo D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Implement a stack using the C++ STL (std::stack).

Source Code:

ILOA > hare_seatwork4p1.cpp > main()

```cpp
int main(){

    // push
    // pop
    //isEmpty
    //isFull
    //Top

    //create an object
    std::stack<char> stack1;

    std::cout<<"Is the stack empty? " <<stack1.empty() <<std::endl;
    stack1.push('D');
    std::cout<<"Current top of stack: " <<stack1.top() <<std::endl;
    stack1.push('R');
    std::cout<<"Current top of stack: " <<stack1.top()<<std::endl;
    stack1.push('E');
    std::cout<<"Current top of stack: " <<stack1.top()<<std::endl;
    stack1.push('I');
    std::cout<<"Current top of stack: " <<stack1.top()<<std::endl;
    std::cout<<"Is the stack empty? " <<stack1.empty()<<std::endl;
    std::cout<<"What's the size of the stack: " <<stack1.size()<<std::endl;

    std::stack<char> stack2;

    std::cout<<"Is the stack empty? " <<stack2.empty() <<std::endl;
    stack2.push('D');
    std::cout<<"Current top of stack: " <<stack2.top() <<std::endl;
    stack2.push('A');
    std::cout<<"Current top of stack: " <<stack2.top()<<std::endl;
    stack2.push('D');
    std::cout<<"Current top of stack: " <<stack2.top()<<std::endl;
    std::cout<<"Is the stack empty? " <<stack2.empty()<<std::endl;
    std::cout<<"What's the size of the stack: " <<stack2.size()<<std::endl;

```

```
39      // swap
40      std::cout << "\nBefore swap:" << std::endl;
41      std::cout << "Stack1 top: " << stack1.top() << std::endl;
42      std::cout << "Stack2 top: " << stack2.top() << std::endl;
43
44      stack1.swap(stack2);
45
46      std::cout << "After swap:" << std::endl;
47      std::cout << "Stack1 top: " << stack1.top() << std::endl;
48      std::cout << "Stack2 top: " << stack2.top() << std::endl;
49
50      // emplace
51      stack1.emplace('X');
52
53      std::cout << "After emplace('X') into Stack1:" << std::endl;
54      std::cout << "Stack1 top: " << stack1.top() << std::endl;
55      std::cout << "Stack1 size: " << stack1.size() << std::endl;
56
57      return 0;
58  }
```

Output:

```
Current top of stack: E
Current top of stack: I
Is the stack empty? 0
What's the size of the stack: 4
Is the stack empty? 1
Current top of stack: D
Current top of stack: A
Current top of stack: D
Is the stack empty? 0
What's the size of the stack: 3

Before swap:
Stack1 top: I
Stack2 top: D
After swap:
Stack1 top: D
Stack2 top: I
After emplace('X') into Stack1:
Stack1 top: X
Stack1 size: 4
```

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack
In this section, we will have an implementation of stack through an array and a linked list.

ILO B.1: Implement a stack using a Static Array.

ILOB1 > hare_seatwork4p1p2.cpp > main()

```cpp
#include <iostream>

// Global declaration
#define maxSize 10

int stackArr[maxSize];
int top = -1;
int i, newData;

// Function prototypes

// Observer operators
bool isEmpty();
bool isFull();
void Top();
void displayAll();

// Transformer operators
void push();
void pop();

int main() {
    int choice;

    while (true) {
        std::cout << "========================";
        std::cout << "      STACK MENU\n";
        std::cout << "========================";
        std::cout << "1. Push";
        std::cout << "2. Pop";
        std::cout << "3. Is Empty";
        std::cout << "4. Is Full";
        std::cout << "5. Top";
        std::cout << "6. Display All";
        std::cout << "7. Exit";
        std::cout << "Enter your choice: ";
        std::cin >> choice;
```

```
        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                std::cout << "Is the stack empty? "
                          << (isEmpty() ? "Yes" : "No") << std::endl;
                break;
            case 4:
                std::cout << "Is the stack full? "
                          << (isFull() ? "Yes" : "No") << std::endl;
                break;
            case 5:
                Top();
                break;
            case 6:
                displayAll();
                break;
            case 7:
                std::cout << "Exiting program..." << std::endl;
                return 0;
            default:
                std::cout << "Invalid choice!" << std::endl;
        }
    }

    return 0;
}
```

```
71  // Observer operators
72  bool isEmpty() {
73      return (top == -1);
74  }
75  bool isFull() {
76      return (top == maxSize - 1);
77  }
78
79  void Top() {
80      if (isEmpty()) {
81          std::cout << "Stack is empty!" << std::endl;
82          return;
83      }
84
85      std::cout << "Top element: " << stackArr[top] << std::endl;
86  }
87
88  void displayAll() {
89      if (isEmpty()) {
90          std::cout << "Stack is empty!" << std::endl;
91          return;
92      }
93
94      std::cout << "Stack contents (Top to Bottom): ";
95
96      for (i = top; i >= 0; i--) {
97          std::cout << stackArr[i] << " ";
98      }
99
100     std::cout << std::endl;
101 }
102
103 // Transformer operators
104
105 void push() {
106     if (isFull()) {
107         std::cout << "Stack Overflow! Cannot push another element."
108                   << std::endl;
109         return;
110     }
```

```
112     std::cout << "Enter a value to push: ";
113     std::cin >> newData;
114 
115     stackArr[++top] = newData;
116 
117     std::cout << newData << " pushed successfully." << std::endl;
118 }
119 
120 void pop() {
121     if (isEmpty()) {
122         std::cout << "Stack Underflow! Stack is empty."
123                   << std::endl;
124         return;
125     }
126 
127     std::cout << "Popped element: " << stackArr[top] << std::endl;
128     top--;
129 }
```

B.2. Stacks using Linked Lists

Tasks:

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

Displaying all elements

```
// Display all
template <typename T>
void displayAll() {
    if (isEmpty<T>()) {
        std::cout << "Stack is empty" << std::endl;
        return;
    }


    std::cout << "Displaying the stack: \n";
    Node<T>* temp = head<T>;
    while (temp != nullptr) {
        std::cout << "\n| " << temp->data << " | ";
        temp = temp->next;
    }
    std::cout << std::endl;
}
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
PS C:\Users\TIPQC\Documents\CPE010_HOA4p1_Hare> C:\Users\TIPQC\Documents\CPE010_HOA4p1_Hare\mainhare.exe
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ----
Top of stack: e

 ----- Display all the elements -----
Displaying the stack:

| e |
| p |
| c |

 ----- Popping all the elements -----
 Popped: e
 Popped: p
 Popped: c

 ----- Testing the Underflow -----
Stack underflow.
PS C:\Users\TIPQC\Documents\CPE010_HOA4p1_Hare>
```

Description for each operation:

1. push()
It adds a new element to the top of the stack. In the program, 'c', 'p', and 'e' are pushed, that makes 'e' be the top element.

2. pop()
It removes and returns the top element of the stack. The elements are removed in the order 'e', 'p', and 'c' that follows LIFO.

3. displayAll()
It displays all elements in the stack from top to bottom. The output is 'e', 'p', and 'c', it follows the LIFO order.

4. top()
It displays the top element without removing it. The output shows 'e' because it was the last element added.

5. isEmpty()
It checks if the stack is empty. It is used to stop the program from popping elements once the stack has no more items.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:
1. Create a Stack.
2. While(end of input is not reached) {
   a) If the character read is not a symbol to be balanced, ignore it.
   b) If the character is an opening symbol, push it onto the stack.
   c) If it is a closing symbol:
      i) Report an error if the stack is empty.
      ii) Otherwise, pop the stack.
   d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

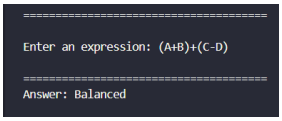
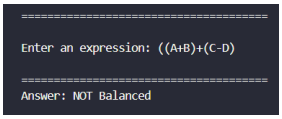
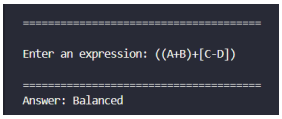
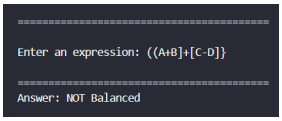
Tools Analysis:
● How do the different internal representations affect the implementation and usage of the stack?

Answer:

The array and linked list stack is storing elements in different way, but both still do the same operations like push, pop, and top, and still follow LIFO. The array implementation is more easy to make because it use fixed size array, but it has limit and cannot store too many elements. While the linked list implementation is more flexible because it use nodes and dynamic memory, so it can grow when more data is added and no fixed size limit. In this activity, both array and linked list can check if the expression is balanced or not, but the way they store data is different. array is more simple, but linked list is better when data size is not fixed.

T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

Self-Checking:
For the following cases, complete the table using the code you created.

Expression	Valid (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Every opening parenthesis has a matching closing parenthesis, so the expression is balanced.
((A+B)+(C-D)	N		One opening parenthesis does not have a matching closing parenthesis, making the expression not balanced.
((A+B)+[C-D])	Y		The parentheses and square brackets are correctly matched and properly nested, so the expression is balanced.
((A+B]+[C-D]}	N		The opening and closing symbols do not match correctly, so the expression is not balanced.

Conclusions/Learnings:

Provide the following:
- Summary of lessons learned

I learned how the stack data structure works using the LIFO principle. I was able to implement a stack using both arrays and linked lists and perform the basic operations such as push, pop, top, and checking if the stack is empty or full. I also learned how stacks can be applied to solve real problems, such as checking whether an expression has balanced symbols. This lesson helped me understand that different implementations can perform the same operations while using different ways to store data.

● Analysis of the procedure

The procedure was easy to follow because there's already a source code that was given to use and it introduced the stack operations first before applying them to a real problem. Implementing the stack using arrays and linked lists helped me understand the differences between the two approaches.

● Analysis of the supplementary activity

It allowed me to apply the stack operations that I learned by checking whether expressions were balanced. It showed how the push and pop operations work together to compare opening and closing symbols correctly. By testing different expressions, I was able to identify which ones were balanced and which ones were not. It helped me understand that stacks are useful not only for storing data but also for solving practical programming problems.

● Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for Improvement?

I think I did just fine in this activity because I was able to implement the required stack operations and complete both the array and linked list implementations. I also successfully applied the stack to solve the balanced symbols problem and tested my program using different expressions. One area I can still improve is writing my code more efficiently and becoming more familiar with debugging errors because I'm getting problems figuring it out more often.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot, Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Since I'm not so good at constructing words, I tend to rely on grammar checkers to easily explain my answers.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.